

Checkpoint 1

1. Project Title: *KickMetrics*

2. Group Members:

- a. Aryan Garg (garg46@wisc.edu)
- b. Jeremy Jarocki (jjjarocki@wisc.edu)
- c. Kaeya Kapoor (nkapoor5@wisc.edu)

3. Motivation and Project Description:

Our project focuses on building a soccer analytics application that helps users explore and compare player performance data across Europe's top football leagues. The application will be a standalone tool built with a React frontend and a Python backend connected to a MySQL database. A relational database is essential here because soccer data is inherently interconnected -- players belong to clubs, clubs compete in leagues, players appear in matches, and their market valuations change over time. Tracking all of these relationships accurately and querying across them requires a structured relational system.

The application is designed for a few different types of users. Team scouts and general managers could use it to evaluate potential signings by filtering players based on stats like goals, assists, and minutes played, or by comparing market value trends. Fantasy football users could browse player performance data to make informed picks for their teams. Casual fans could also use the app to look up their favorite players, see how they've performed across seasons, and explore head-to-head comparisons.

At a minimum, the application will support inserting, updating, deleting, and querying player and match data. Users will be able to search for players by name, position, club, or nationality, view detailed match appearance histories, compare two players side by side, and explore transfer histories and market value changes over time. All queries will pull from multiple related tables to present meaningful, connected information rather than isolated stats.

4. Dataset Plan:

Our primary dataset is the "Football Data from Transfermarkt" hosted on Kaggle (<https://www.kaggle.com/datasets/davidcariboo/player-scores>). This is a well-maintained, automatically updated dataset sourced from Transfermarkt.com that includes over 37,000 players, 80,000+ matches, 1,800,000+ player appearance records, 520,000+ market valuation entries, 99,000+ transfer records, and 1,100,000+ game events such as goals, cards, and substitutions. The data is broken into clean CSV files for players, clubs, competitions, games, appearances, game events, player valuations, and transfers, which map directly onto the entity sets in our database design. The appearances table alone has over 1.8 million rows, which comfortably exceeds the 100,000-record requirement. Key attributes include player biographical info (name, date of birth, position, nationality), match details (date, scores, competition), and per-game stats (goals, assists, minutes played, cards).

Our secondary dataset is the "Football Players Stats 2025-2026" from Kaggle (<https://www.kaggle.com/datasets/hubertsidorowicz/football-players-stats-2025-2026>), sourced from FBref. This dataset provides detailed seasonal aggregates for players across the Big 5 European leagues, with over 250 columns covering advanced metrics like passing accuracy,

progressive carries, shot-creating actions, and defensive stats. We plan to use this as an enrichment source to supplement the Transfermarkt data with deeper per-season performance breakdowns that are not available in the primary dataset.

5. Initial ER Diagram and Description:

Entity Sets (9 total):

Player -- player_id (PK), name, date_of_birth, position, sub_position, foot, height, country_id (FK)

Club -- club_id (PK), name, stadium_name, squad_size, country_id (FK), domestic_competition_id (FK)

Competition -- competition_id (PK), name, type (league/cup/international), country_id (FK)

Game -- game_id (PK), date, season, round, home_club_id (FK), away_club_id (FK), home_score, away_score, competition_id (FK), stadium

Appearance -- appearance_id (PK), player_id (FK), game_id (FK), club_id (FK), minutes_played, goals, assists, yellow_cards, red_cards (this is the 1.8M+ record table)

GameEvent -- event_id (PK), game_id (FK), player_id (FK), club_id (FK), event_type (goal/card/substitution), minute, description

PlayerValuation -- valuation_id (PK), player_id (FK), date, market_value_eur, club_id (FK)

Transfer -- transfer_id (PK), player_id (FK), from_club_id (FK), to_club_id (FK), transfer_date, transfer_fee, season

Country -- country_id (PK), name, confederation

Relationship Sets (11 total):

Player belongs_to Club (Many-to-One) -- Each player has one current club, and a club has many players. Total participation on the Player side.

Player appears_in Game (Many-to-Many, via Appearance) -- A player can appear in many games, and a game has many players. This is the required M:N relationship.

Game part_of Competition (Many-to-One) -- Each game is part of one competition, and a competition has many games. Total participation on the Game side.

Club competes_in Competition (Many-to-Many) -- A club can play in multiple competitions (league + cup), and a competition has many clubs.

Player has_valuation PlayerValuation (One-to-Many) -- A player has many valuation records over time. Total participation on the PlayerValuation side.

Player has_transfer Transfer (One-to-Many) -- A player can have multiple transfers throughout their career.

Transfer from_club Club (Many-to-One) -- Each transfer has one source club that the player is leaving.

Transfer to__{club} Club (Many-to-One) -- Each transfer has one destination club that the player is joining.

Game has__{event} GameEvent (One-to-Many) -- Each game produces multiple events (goals, cards, substitutions). Total participation on the GameEvent side.

Player from__{country} Country (Many-to-One) -- Each player has one nationality, and a country has many players.

Club located__{in} Country (Many-to-One) -- Each club is based in one country, and a country can have many clubs.

Competition held__{in} Country (Many-to-One) -- Each competition belongs to one country, and a country can host multiple competitions.

Design Assumptions:

Appearance is modeled as an associative entity that resolves the many-to-many relationship between Player and Game. It carries its own attributes like goals, assists, minutes played, and cards received, which makes it an entity in its own right rather than just a junction table.

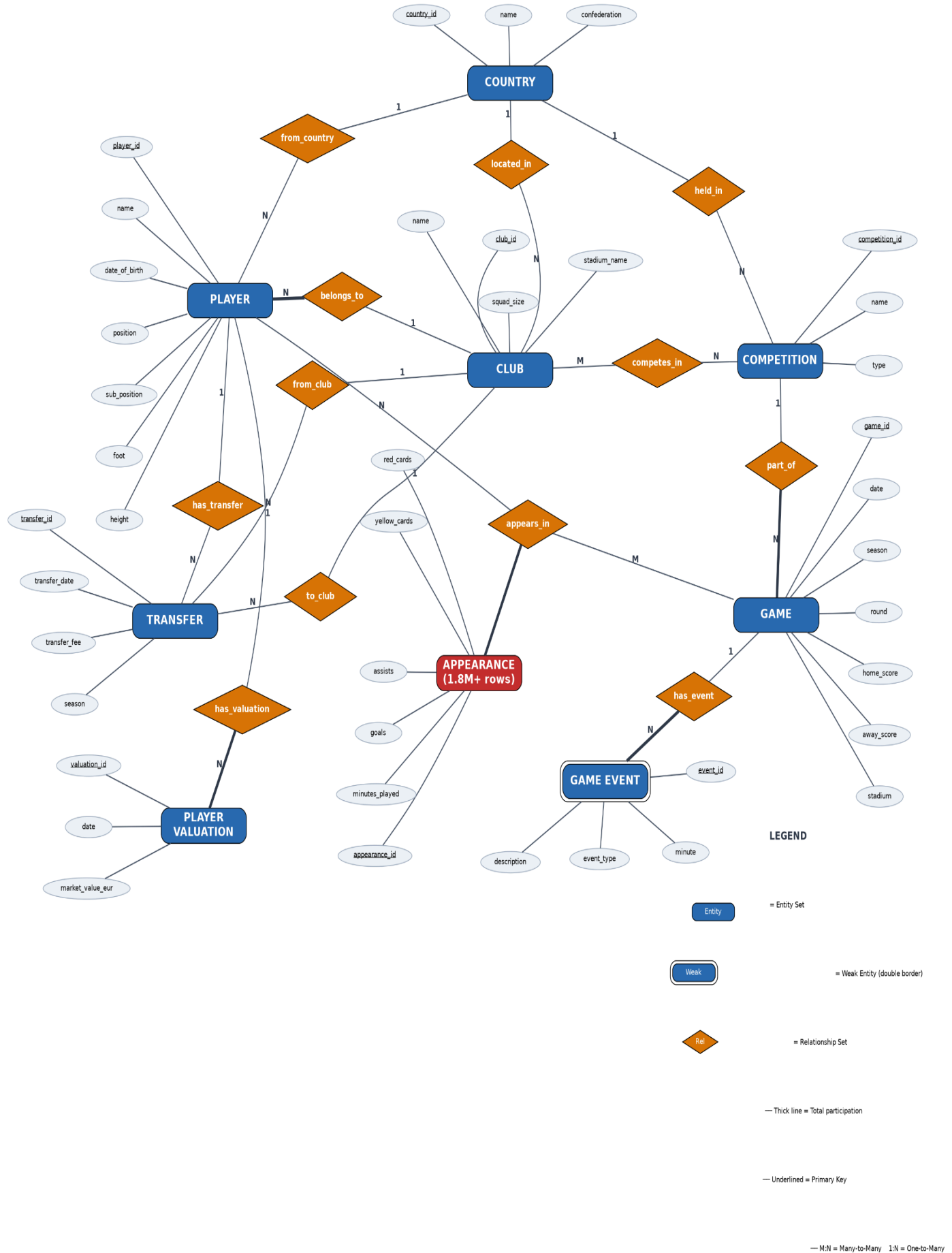
A player's current club is tracked through a foreign key in the Player table, but their full club history is captured through the Transfer and Appearance tables. This means we can see both where a player is now and everywhere they have been.

GameEvent is a weak entity that depends on Game for identification. A game event only exists in the context of a specific match, and its event__{id} is only meaningful when paired with a game__{id}.

Each Transfer links to two separate Club relationships -- from_club and to_club -- representing the selling and buying clubs. This lets us query transfer activity from either side (which clubs are buying the most, which clubs are selling, etc.).

PlayerValuation tracks market value snapshots over time, with each record stamped with a date. This makes date a meaningful query attribute as required by the project spec, enabling queries like "how did a player's value change after a transfer" or "show me all players whose value peaked in 2023."

Country serves as a shared reference entity linked to Player (nationality), Club (location), and Competition (host country). This avoids duplicating country data across multiple tables.



Written description to accompany the diagram:

The core of our database design revolves around the Player, Club, and Game entities. Players belong to clubs and appear in games, forming the central many-to-many relationship that gets resolved through the Appearance entity. Appearance works as both a relationship and an entity in its own right because it carries important attributes like goals scored, assists, minutes played, and cards received in each match. This is also our largest table at over 1.8 million records, which gives us plenty of data for meaningful performance analysis and indexing work.

Games are linked to Competitions through a `part_of` relationship, where each competition represents a league or tournament. Clubs participate in multiple competitions (for example, a Premier League club might also play in the Champions League), creating another many-to-many relationship. The GameEvent entity captures specific in-match incidents like goals, yellow and red cards, and substitutions. It is modeled as a weak entity since an event only makes sense within the context of a particular game.

On the player profile side, PlayerValuation tracks how a player's market value changes over time, with each record tied to a specific date. This supports time-based queries that are central to our application. The Transfer entity records player movements between clubs, linking to two separate Club relationships (`from_club` and `to_club`) along with transfer fees and dates. Country acts as a shared reference entity that connects to players through nationality, to clubs through location, and to competitions through their host country, keeping the design normalized and avoiding repeated data.

6. Technical Plan:

We will use MySQL as our database management system since it is the recommended DBMS for this course and fits well with the scale of our data. The backend will be written in Python, using Flask as the web framework to handle API routes and database connections. We plan to use the mysql-connector-python library to interface with MySQL from Python. For the frontend, we will build a single-page application using React, which will communicate with the Flask backend through REST API calls. We may also use libraries like Recharts or Chart.js within React for data visualization features such as player comparison charts and market value trend graphs.

7. Project Management Plan:

Member	Week 3 Tasks	Week 4 Tasks	Week 5 Tasks	Week 6 Tasks	Week 7 Tasks	Week 8 Tasks
Aryan	Design relational schema, create MySQL tables, load datasets	Normalize tables, build data dictionary	Write stored procedures and complex queries	Create indexes, run performance tests (before/after)	Finalize DB optimizations, help with integration testing	Final report (DB sections), presentation prep
Jeremy	Help refine ER diagram	Start barebones React UI (search, filters, navigation)	Build out player comparison and stats display pages	Polish UI, add data visualization (charts/graphs)	Connect frontend to backend API endpoints	Final UI cleanup, presentation prep
Kaeya	Help with data cleaning and CSV preprocessing	Set up Flask backend, write initial API routes	Build API endpoints for stored procedures and views	Connect backend to frontend, handle query logic	End-to-end testing, bug fixes	Final report (backend sections), presentation prep

8. AI-Use Statement:

- Our team used Claude (Anthropic) as an AI tool for this checkpoint.
- Used for brainstorming the project direction within soccer statistics, finding datasets that meet the 100k+ record requirement, and suggesting tech stack options.
- Claude recommended the Transfermarkt and FBref datasets on Kaggle. We verified both on Kaggle ourselves to confirm record counts and attribute relevance before accepting.
- Tech stack suggestions aligned with what we had already planned (MySQL, Python, React), so those were accepted as-is.
- Claude helped draft the initial ER diagram layout and entity/relationship structure. We reviewed all entities and relationships against the actual dataset columns and project requirements before finalizing.